ColdCode: Cold Data Encoding for Enhanced Reliability and Lifetime in 3D NAND Flash

Qiao Li*
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates

Shangyu Wu*
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates

Zheng Wan
School of Informatics
Xiamen University
China

Yufei Cui
School of Computer Science
McGill University
Canada

Jie Zhang
School of Computer Science
Peking University
China

Chun Jason Xue
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates

Abstract

Cold storage, which stores rarely accessed data, dominates modern data centers but is poorly served by NAND flash's data randomization. As a common practice today by flash vendors, data randomization is applied in NAND flash chips to avoid extreme data patterns that generate the worst-case raw bit error rate (RBER). However, this paper demonstrates that data randomization rules out the opportunity to explore data patterns with very low RBERs, through a comprehensive analysis on data randomization in 3D NAND flash chips (across 8 models). Motivated by this, we propose **ColdCode**, a novel data coding framework to replace the conventional randomizer in 3D high-density flash for cold data storage. Using a tag that indicates coldness information passed from the file system to solid-state drive (SSD) controllers, the controller encodes cold data to enhance reliability and extend the lifetime. **ColdCode** employs two coding techniques: skewed coding and reversed Huffman coding, applied based on the data entropy. These techniques effectively reduce the RBER of encoded data compared to conventional randomization. Experimental results on real high-density 3D flash chips show that, under the same error conditions, the skewed coding and reversed Huffman coding reduce the average RBER by 42% and 30%, respectively. Consequently, the lifetime of the flash chips is prolonged by factors of 2.97× and 1.7×, respectively, compared to data randomization.

CCS Concepts: • **Computer systems organization** → **Firmware**; • **Hardware** → **Memory and dense storage**.

Keywords: Flash reliability, Randomization, Data coding, Cold storage

ACM Reference Format:
Qiao Li*, Shangyu Wu, Zheng Wan, Yufei Cui, Jie Zhang, and Chun Jason Xue. 2026. ColdCode: Cold Data Encoding for Enhanced Reliability and Lifetime in 3D NAND Flash. In *European Conference on Computer Systems (EUROSYS '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3767295.3769388

*Both authors contribute equally to this paper.

1 Introduction

3D NAND flash is increasingly favored for cold data storage due to its declining cost per gigabyte and high density [32, 41]. Although hard disk drives (HDDs) have traditionally dominated cold data storage due to lower upfront costs, the total cost of ownership (TCO) for 3D NAND flash is becoming more competitive, driven by savings in power, cooling, and physical space. To reduce the cost per bit, current 3D NAND flash memories stack a great number of layers in a block and many cells in a horizontal wordline to deliver large capacities within a fixed silicon area [16, 38]. However, the complex connection of flash cells in 3D flash blocks introduces new leakage paths of charges stored in flash cells, where the stored data suffers from high raw bit error rates (RBERs) [18, 25, 51]. Furthermore, RBER variations among pages and among layers exacerbate such reliability concerns in modern high-density 3D NAND flash memory [26, 31, 56].

To avoid extreme data patterns with ultra-high RBERs, a randomizer is unanimously applied in today's 3D NAND flash products. Specifically, randomization encodes the original data to even distribution across all voltage states in flash memory cells. For example, 3-bit data is represented as 8 voltage states in triple-level cell (TLC) flash. Such randomization can prevent programming many cells to voltage states that are prone to voltage shifting, thus reducing RBERs.

Existing studies have shown that high voltage states tend to suffer from more voltage shifting [4, 15, 28]. This phenomenon has been well studied in planar flash memories, with various strategies proposed for enhancing reliability. Cai et al. [4, 6] observed that more than 90% bit errors occur due to the voltage shift of the two high voltage states in planar 2-bits-per-cell multi-level cell (MLC) flash memories. Based on this observation, several approaches are proposed to reduce the RBER of flash memories, including flipping the data bits to generate more low-voltage states [13, 14, 23, 44, 57], and eliminating one or several high-voltage states [15, 43, 54]. The reliability variations among voltage states also exist on 3D TLC NAND flash memories [28], and several studies

EUROSYS '26, Edinburgh, Scotland UK

ACM ISBN 979-8-4007-2212-7/26/04
https://doi.org/10.1145/3767295.3769388

adopt the same mapping principle to generate more low-voltage states on 3D TLC flash memories [43, 53]. *However, this paper shows that reducing high-voltage states or increasing low-voltage states does not work well on high-density 3D NAND flash memories.*

This work is the first to conduct a systematic study on the randomizer in 3D NAND flash memory and to demonstrate its impacts on data reliability. By characterizing the error behaviors of randomized data and other constructed data patterns in eight 3D flash chips from four vendors, we present several expected and unexpected observations of reliability behaviors. As expected, the randomizer avoids the occurrence of extremely high RBERs by redistributing the original data evenly across all voltage states. However, randomization proves suboptimal, as we identify specific data patterns that outperform it, especially for cold data. Besides, traditional low-voltage optimizations become counterproductive in dense 3D NAND configurations, sometimes exacerbating RBER higher than randomized data.

To address the vulnerability of cold data to long retention errors, this paper introduces **ColdCode**, a new coding framework to replace the randomizer for cold data in high-density 3D NAND flash. By leveraging the hotness information from the file system [22, 27], ColdCode dynamically selects between two entropy-aware encoding schemes for cold data: (1) *skewed coding* for low-entropy data (e.g., raw data without compression or encryption), which exploits the characteristic that combined characters in the original data present an uneven distribution. (2) *reversed Huffman coding* for high-entropy data (e.g., compressed or encrypted files), which makes a trade-off between physical space and reliability by increasing the coded data length. Both coding mechanisms construct data patterns that lead to low RBERs after a long retention time.

The results on the latest 3D flash chips with 170+ layers reveal that the proposed skewed coding and the reversed Huffman coding can reduce the RBER of cold data by 42% and 30% on average, respectively. The reduction in RBER results in a significant reduction in refresh frequency, which greatly prolongs flash lifetime due to a decrease in write amplification [21, 29]. By conducting simulations with real workloads, the proposed two coding mechanisms can prolong flash lifetime by 2.97 times and 1.7 times, respectively.

The main contributions of this paper are as follows.

- **Characterization Insight:** Through systematic analysis of 3D NAND flash randomizers, we reveal that while randomization prevents worst-case RBER scenarios, it eliminates opportunities to exploit superior low-RBER patterns for cold data;
- **Novel Encoding Framework:** We present entropy-aware encoding schemes, ColdCode, to replace randomization with two specialized schemes for lower RBERs over a long retention time;
- **Real-Device Validation:** Through extensive experiments on cutting-edge 170+ layer 3D flash chips, we demonstrate 42%/30% RBER reductions and 2.97×/1.7× lifetime extensions.

The remainder of this paper is organized as follows. Section 2 presents the background and related work. Section 3 presents the real-device characterization. The proposed coding design is presented in Section 4. Evaluations are presented in Section 5. Section 6 concludes this paper.

2 Background and Related Work

2.1 Cold Storage with NAND Flash

3D NAND flash is increasingly being adopted for cold storage due to its declining cost per gigabyte, high density, and energy efficiency compared to HDDs. While HDDs remain cost-advantageous upfront for cold storage, flash's total cost of ownership (including power, cooling, and space) is now competitive. Flash also delivers faster access and higher reliability, benefiting certain cold storage workloads, such as archival and compliance-driven storage. With advancements like QLC NAND and high-capacity SSDs, flash's role in cold storage continues to expand, motivating us to optimize 3D NAND flash behavior for these scenarios.

A 3D NAND flash memory chip contains multiple dies, each of which is composed of multiple planes. A plane contains thousands of flash blocks, which are three-dimensional arrays of flash cells, as shown in Figure 1. Read and write operations in 3D NAND flash are conducted on a wordline (WL), as the control gates of all cells in the horizontal direction are connected. A WL could contain millions of flash cells for high densities. By storing a certain amount of charge, flash cells encode data bits into multiple levels of threshold voltage (V_{th}). The V_{th} distribution of triple-level cell (TLC) flash memory in Figure 1 is the most popular type in 3D NAND memories. The most/center/least bits in a WL form MSB/CSB/LSB pages, respectively.

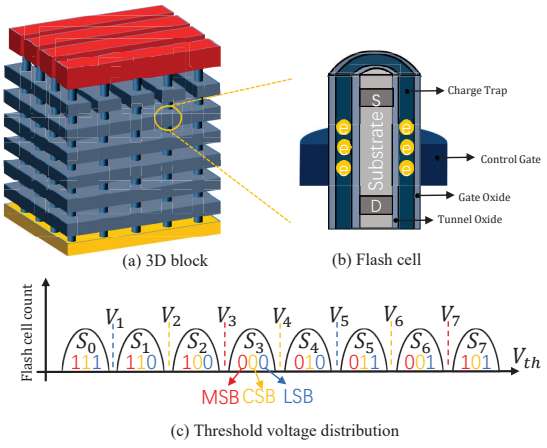


Figure 1. 3D NAND blocks, cells and the V_{th} distribution.

Due to various forms of disturbances and noise, the V_{th} of flash cells may become biased, introducing bit errors in the stored data. During programming and erase (P/E) operations, the high voltages damage flash cells' capability of storing charges, thus resulting in increased RBERs in stored data after repeated P/E cycles. Additionally, read and program operations slightly increase the V_{th} of neighboring flash cells inside the same flash block, when extra charges could be unintentionally injected into the disturbed cells. Over the retention period, the charges slowly leak out of flash cells, which is the major error source in 3D NAND flash memories. The leakage rate is closely related to the amount of charges, where the leakage is faster in cells with higher threshold voltages. As the density and capacity scale, these error sources introduce higher error rates to 3D NAND flash than ever. To address reliability concerns, prior research has introduced various strategies. This study investigates how data coding affects the reliability of 3D NAND flash memory.

2.2 Data Coding for Flash Cells

NAND flash memories store charges in cells to represent the stored data. Voltage levels representing different data naturally have various capabilities for maintaining charges. This phenomenon has been well studied in planar flash memories, followed by some strategies proposed to explore the characteristics for reliability enhancement. These methods have recently been applied in 3D NAND flash memories.

One strategy is to eliminate some of the high voltage states that suffer from high RBERs [7, 15, 43, 46]. In TLC flash, there are 8 voltage states to store 3 bits of data. Errors mainly occur at the read voltage between pairs of high voltage states. Eliminating one or several of the high voltage states could prevent the high RBERs from happening, but at the sacrifice of the storage capacity, which is usually adopted in the late life stage of NAND flash memory. Another strategy is to produce more reliable states by using certain mapping strategies. The rationale behind this set of methods is rooted in the observation that high voltage states are susceptible to significant retention loss, which serves as the primary source of errors in both planar and 3D NAND flash memories [5, 56]. To generate more low voltage states, multiple strategies were proposed [53, 59]. Since the lowest voltage state represents '11' in MLC flash and '111' in TLC flash, increasing the population of 1 in the encoded data could result in more low voltage states [13, 14, 23, 44, 57]. This set of methods requires recording the flag whether the bits are flipped, which is prohibitively high, especially in the granularity of several bits. In addition, several studies propose to apply Huffman coding in flash memory. PDLCS [42] dynamically changes the V_{th} distribution for data privacy in 3D flash. Ahrens et al. [2] adopt Huffman coding for compression to achieve more space for error correction parity to improve the error correcting capability. DHC [45] designs Huffman coding based on frequencies to reduce the number of voltage states.

In this work, we conduct a thorough study on different coding strategies and analyze their impacts on data reliability. We will show expected observations, e.g., high voltage states do suffer from more errors, and unexpected observations, e.g., mapping data to low voltage states does not always enhance the data reliability on 3D NAND.

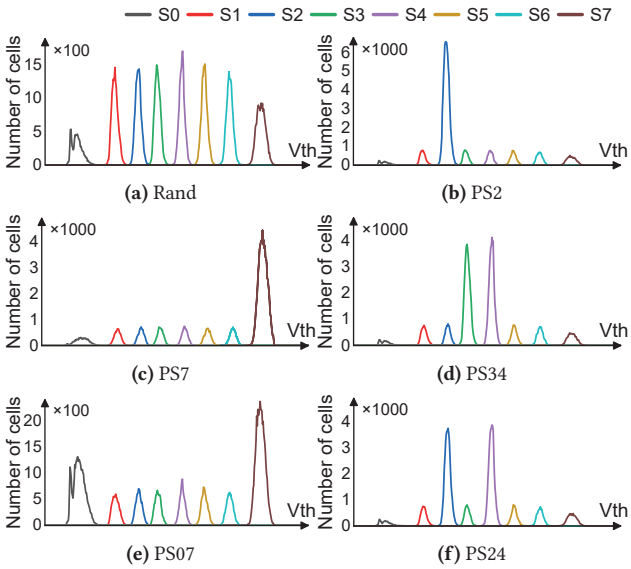


Figure 2. The threshold voltage distribution of data patterns.

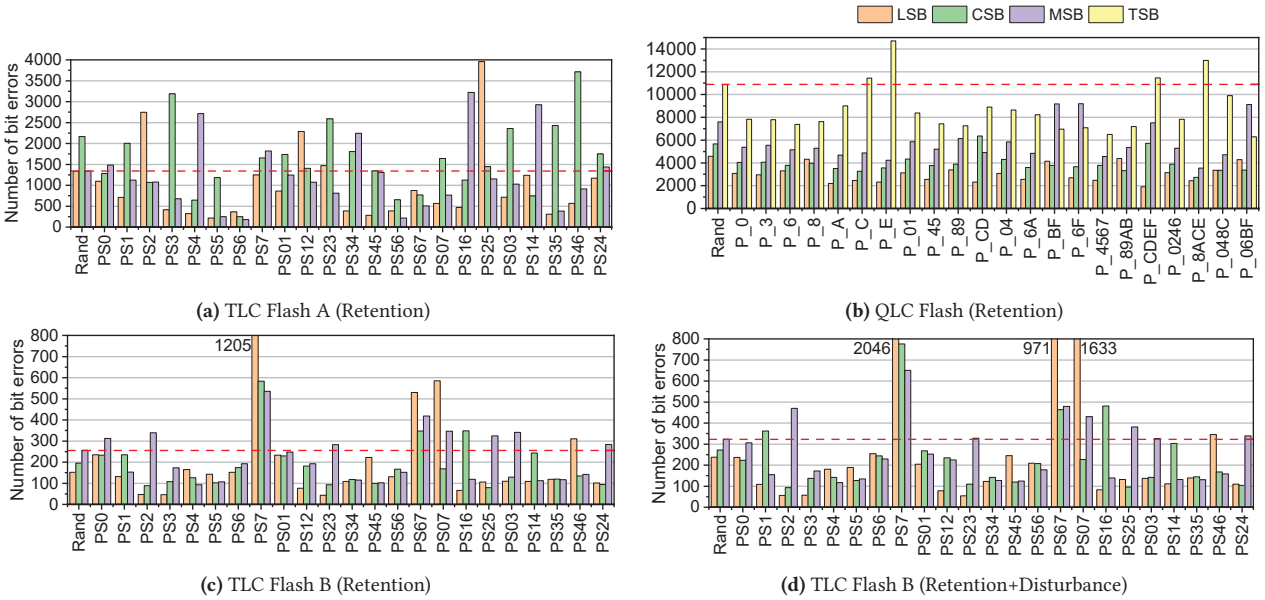
3 Characterization of Randomizer

3.1 Characterization Methodology

We conduct the experiments on YEESTOR flash testing platform, which serves as a flash controller to manage NAND flash chips [50]. The platform sends basic flash operation commands and collects the RBER from readout data. The evaluations are conducted on eight flash chips from four vendors, including five TLC and three QLC flash chips. Although each chip type has its own unique properties, the same conclusions were drawn for all of them. Due to the page limit, we mainly present data from three representative chips and detail the observations from one TLC flash chip.

We constructed 24 data patterns for TLC flash as follows, which are grouped into three categories.

- Rand: Data is randomized and is evenly distributed among 8 voltage states, as shown in Figure 2a. The randomizer is implemented by a linear feedback shift register, which shifts the bits one position to the right and replaces the vacated bit with the XOR of the bit shifted. The original information bits would be transformed bit by bit based on a polynomial function. This is widely applied in real NAND flash chips to avoid significant RBER variation among data.



(a) TLC Flash A (Retention)

(b) QLC Flash (Retention)

(c) TLC Flash B (Retention)

(d) TLC Flash B (Retention+Disturbance)

Figure 3. Error comparison of different coding strategies. The dashed lines present the bit errors of MSB page in *Rand*.

- `Single State (SS)`: $\frac{9}{16}$ of the data will be programmed to one of the 8 voltage states ($S_i, 0 \le i \le 7$), and $\frac{7}{16}$ of data is evenly distributed among other 7 voltage states, as shown in Figures 2b to 2c. We use `PSi` to represent these patterns.
- `Double State (DS)`: $\frac{5}{8}$ of data is programmed to S_i and S_j, evenly distributed among these two states. The remaining $\frac{3}{8}$ of data is evenly distributed among the other 6 voltage states, as shown in Figures 2d to 2f. `PSij` is used to represent these data patterns.

For QLC flash, we use 0 to F to label 16 voltage states from low to high and construct data patterns dominated by specific voltage states. For example, `P_06BF` represents the pattern that has lots of voltage states indexed with 0 (the lowest state), 6, B, and F (the highest state). In the following evaluations, a total of 24 different data patterns are implemented for comparison. We select flash blocks with similar reliability strength to avoid bias from process variations. Each data pattern is programmed into two blocks to further reduce the experimental bias.

3.2 RBERs of Flash Pages

We present several observations on the RBERs in the granularity of pages, where we observe both expected and unexpected error behaviors. Figure 3 shows the average RBERs of all WLs in a 3D NAND block. We perform 1000 and 100 P/E cycles on TLC and QLC flash blocks, respectively, then bake the flash chips under 100°C for 2 hours, to simulate the retention loss (Figures 3a- 3c), and conduct 300 block read operations to introduce read disturbance (Figure 3d).

Observation 1: Compared to `Rand`, some data patterns have higher RBER, and some have lower RBER. This observation can be made in all eight evaluated flash chips, while the patterns that have lower/higher RBER are not the same. Figure 3 illustrates three of them, where each chip presents different degrees of variations among patterns. The differences among chips could be caused by multiple aspects, such as the manufacturing process of vendors, the design of programming algorithms, and so on. In the following, we discuss the data collected on TLC flash B in detail.

Specifically, three single-state strategies in Figure 3c have higher RBERs than `Rand`, including `PS0`, `PS2`, and `PS7`. Especially for `PS7`, which has lots of high voltage state S7, the LSB RBER is more than 4 times of `Rand`, while other single-state patterns have lower RBERs than `Rand`. The extremely high RBER in `PS7` is not surprising, because lots of cells are programmed to S7, and they tend to suffer from severe retention loss. However, what is unexpected is that the good data patterns (RBERs lower than `Rand`) are not those dominated by low voltage states, but those dominated by medium voltage states. This observation also stands in Figure 3d, where flash chips have endured retention loss and read disturbance. Their combined effect leads to a wider voltage distribution with worse reliability [36], as the complementary effects of charge loss (retention) and gain (disturbance) are not perfectly balanced. The RBERs in Figure 3d significantly increase compared to those in Figure 3c. Considering the impacts of retention time and read disturbance, the RBER differences among different data patterns become even larger. For example, the RBER increase in `PS7`, whose RBER after retention is already very high, is much more significant than others.

For double-state coding, three `DS` strategies (`PS34`, `PS56`, and `PS35` in Figure 3c) have much lower RBERs than `Rand`. Similarly, those patterns dominated by S7 have higher RBERs

and those dominated by low voltage states might also have high RBERs. For example, all three pages in PS01 have comparable RBERs to the MSB of Rand. This observation is also contrary to the common belief that mapping data to low voltage states will improve the reliability [23, 44, 57]. The above analysis explains why randomizer is widely applied in current 3D NAND flash memory chips. As user data could have very different distributions, the adoption of the randomizer avoids extreme cases with high RBERs, such as patterns dominated by S7. Nevertheless, it rules out the opportunities to explore lower-RBER patterns.

Observation 2: The variation among the three pages in Rand of TLC flash, which is universal in all flash memories, could be relieved by some data patterns. Due to the RBER variations among different voltage states, the three pages of Rand usually have different RBERs [3]. This characteristic is common in 3D NAND flash, which limits flash reliability because the worst-case RBER usually defines the design of reliability management. Other than Rand, Figure 3 shows that RBER variations among three pages exist in most coding strategies. The page that has the highest RBER among the three pages could also be different. For example, the MSB page has the highest RBER in Rand, while the LSB page has the highest RBER in PS7. However, one SS strategy PS6, and two DS strategies, PS34 and PS45, result in relatively balanced RBERs among the three pages. We should also notice that it is impossible to guarantee that three pages always have the same RBERs under any situation, as the impacts of error sources on different voltage states are different. In this work, we evaluated flash chips under different retention times and different read disturbances and found that these two patterns (PS34 and PS45) in TLC flash B perform better than other patterns in all cases, though small variations could still exist in some cases.

3.3 Shift of Voltage States

In 3D NAND flash memories, the V_{th} of some cells crossing the read voltages introduces bit errors by causing misreading of the cells. To explain the error behaviors in different data patterns, we examine the voltage state shift using TLC flash B data. We show how flash cells influence each other's threshold voltage (V_{th}), explaining that reducing high-RBER voltage states doesn't lower the overall RBER in high-density 3D NAND flash memories.

Between each pair of adjacent voltage states, one read voltage (V1 to V7) is applied to separate the two states. Error bits may occur at each read voltage. We categorize read voltages into three groups for error behavior analysis. Figure 4 shows the number of error bits introduced at each read voltage. The bit errors at V_i fall into two categories: 1) The errors resulting from cells in S_{i-1} being misread as S_i (S_{i-1}-Right); 2) The errors resulting from cells in S_i being misread as S_{i-1} (S_i-Left). Only the errors caused by cells shifting to the neighboring states are considered because shifting to faraway states rarely happens [25, 37]. As the total number of cells in distinct voltage states can vary, we also present the ratio of state errors, denoting the proportion of cells with V_{th} crossing the designated read voltage, in Figure 4. Two key observations emerge from the analysis of bit errors at each read voltage.

3.3.1 Low Read Voltages (V1 and V2).

Observation 3: The significant right shift of low voltage states occurs in data patterns dominated by low voltage states and also in patterns dominated by high voltage states. The RBER at low read voltages mainly comes from the right shift of low voltage states. Because of that, the data patterns dominated by low voltage states, like PS0 and PS1, have more bit errors at low read voltages, and the ratios of right shift are high. For example, the right-shift ratio of S1 state in PS1 is 1.6%, and PS1 has lots of cells in S1, thus the number of errors at V2 is as large as 162. However, the unexpected behavior is that the data patterns dominated by high voltage states (like PS7 and PS07) could have an even higher ratio of right shift than PS1. As shown in Figure 4, PS7 is the pattern with the largest number of bit errors at V1 and V2. WLs with many high voltage states exhibit high voltage values in all states. When these high-voltage states shift left, the lost charges could easily enter flash cells with low voltages. As a result, lots of low-voltage cells in patterns like PS7 would suffer from severe right shift.

What might not be expected is that the right shift of S1 looks more significant than S0. This is because we can only count the cells that cross the read voltage with bit errors. S0 has a very wide distribution and is far away from the read voltage V1. It is more difficult for S0 to cross the read voltage caused by the voltage increase. In practice, the voltage increase of S0 is more significant than S1, but the increase does not introduce as many bit errors as the increase in S1.

3.3.2 High Read Voltages (V6 and V7).

Observation 4: The significant left shift of high voltage states occurs in data patterns dominated by high voltage states, also in patterns dominated by low voltage states. This observation is similar to Observation 3 regarding low read voltages. According to existing work [4], we know that higher voltage states suffer more from retention errors. This is because they have more charges, and thus, the leakage speed is higher, leading to more charge leakage. Therefore, retention errors mainly occur at high read voltages, e.g., V7 to separate S6 and S7. The three patterns with massive cells in S7 (PS7, PS67 and PS07) are exactly the top 3 patterns with the largest number of error bits at read voltage V7, which is not surprising. The high error rate at V7 is mainly because of the left shift of S7, as shown in Figure 4f. The unexpected behavior is that the data patterns dominated by low voltage states also have a very high ratio of S7 left shifting, e.g., pattern PS0. The reason is that low voltages in PS0 introduce more chances for high-voltage states to

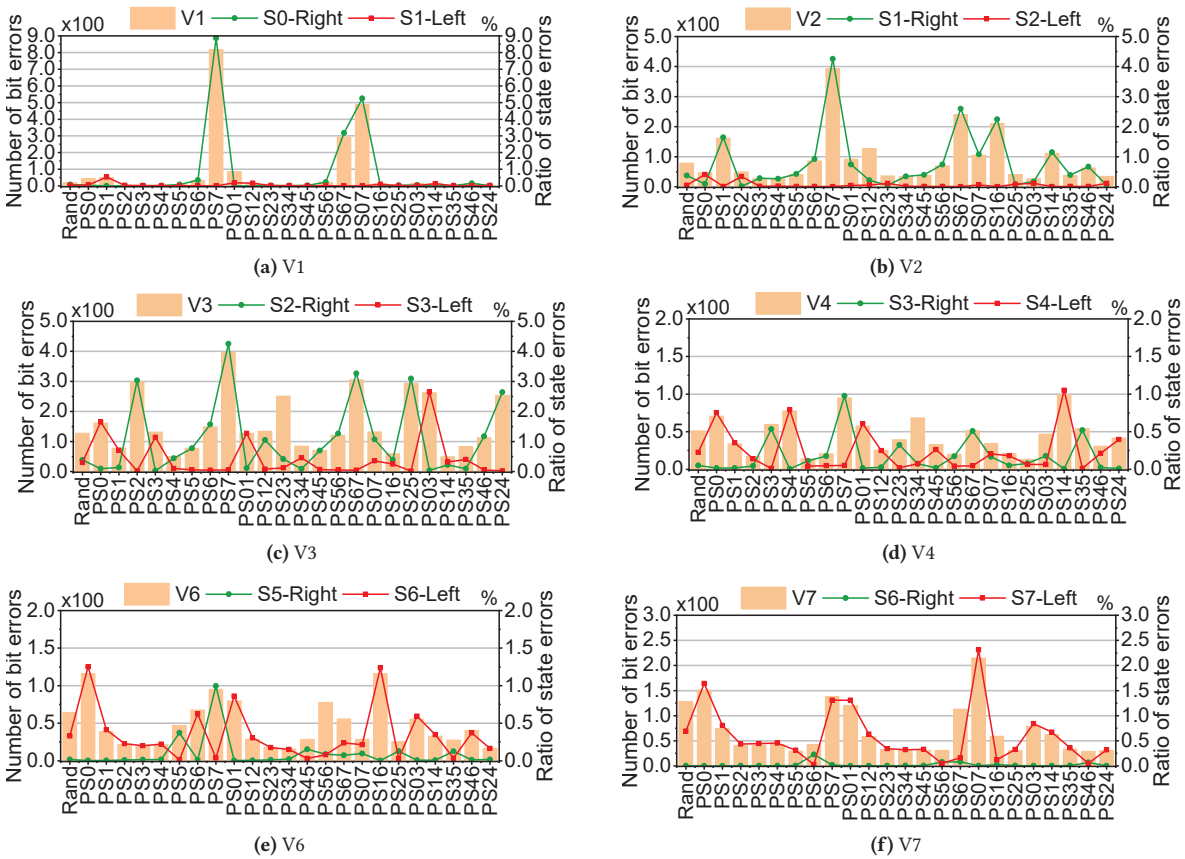


Figure 4. The number of bit errors at each read voltage and the percentages of left-shift and right-shift errors.

leak charges. This could be verified by PS07, which has the highest ratio of S7 left shifting. It has many S0 and many S7, thus providing the fastest leakage path from high voltage states to low voltage states.

3.3.3 Medium Read Voltages (V3, V4 and V5). Medium read voltages exhibit dominant left-shift errors in some data patterns and right-shift errors in others. We use V4 as an example for discussion. Errors at V4 consist of right-shift errors in S3 and left-shift errors in S4. The top 4 patterns with high right-shift errors of S3 are PS3, PS7, PS67 and PS35. PS3 and PS35 contain lots of cells in S3, thus the possibility for S3 to shift is high. PS7 and PS67 contain lots of high voltage states, and the high voltage leads to a high possibility for right shift. The result further validates Observation 3, that cells in a specific state shift to the right either because there are lots of cells in this state or the overall voltage is high. Similarly, the left shift of V4 validates Observation 4, where cells in a state shift to the left either because there are lots of cells in this state or the overall voltage is low.

3.4 Summary of Observations

Drawing from the observations above, data patterns with distinct distributions exhibit markedly different reliability characteristics. Consequently, NAND flash chips are equipped with a randomizer in the controller to randomize all user data before programming it into flash memories. The randomizer transforms the original user data, ensuring an even distribution over 8 voltage states of TLC. The purpose is to avoid extreme data patterns that could introduce high RBERs. However, as different voltage states also have different reliability characteristics, the randomized data suffer from uneven RBERs among three pages in the same WL of TLC or four pages of QLC. The above characterization shows that exploring certain skewed data patterns could achieve lower RBERs than randomized data, as well as balancing RBERs among three pages in the same WL. Furthermore, the existing strategies proposed in the literature on planar flash memories that map high voltage states to low voltage states are not well-suited for high-density 3D NAND flash memories. To address this limitation, this paper develops new coding designs specifically tailored for cold data storage in 3D NAND flash memory.

4 ColdCode: Cold Data Encoding

4.1 Overview

Because cold data are rarely accessed, the RBER should be kept low after a long time, and the low access frequency indicates negligible impacts on the overall performance. Instead of randomizing the data before programming, the main idea is to construct low-RBER data patterns by coding the original data for reliability enhancement. We introduce a collaborative framework where the file system communicates the cold information of files to the SSD controller. The one-bit metadata, indicating whether the file is hot or cold, is passed through extended NVMe commands. Targeting cold data storage, this paper introduces new coding strategies for writing data with a cold tag, while other data will be programmed in the default mode.

Figure 5 shows the overview of **ColdCode**. In commodity SSD devices, the data of write requests is first written to the buffer in SSD controllers. The entropy of written data is then calculated to decide the coding strategy. Our design leverages the processors/cores embedded in SSD controllers to calculate the data entropy in the buffer. By analyzing the data characteristics of different workloads, we classify them into four typical distributions (§ 4.2). For data with low entropy, such as original text and image files, a skewed coding is introduced (§ 4.3), which exploits the uneven distribution of the original data. The skewed coding introduces negligible

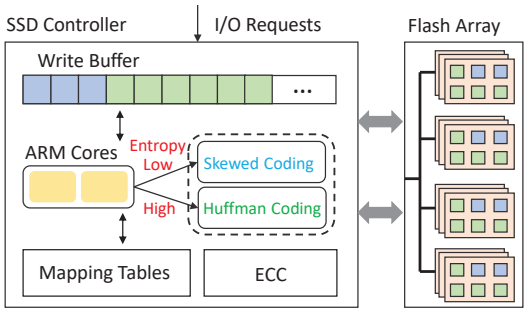


Figure 5. ColdCode overview.

overhead for storing the mapping table. For data with high entropy, such as compressed and encrypted files, there is no skewed property at any granularity. Thus, we propose a reversed Huffman coding (§ 4.4), which trades off storage space for data reliability and flash lifetime. Both coding strategies reduce the RBERs of data stored on 3D NAND, thus increasing the available P/E cycles of flash blocks and also reducing the write amplification caused by periodical data refresh. Although the reversed Huffman coding consumes extra storage space, our evaluation reveals that the benefits brought by RBER reduction significantly surpass the penalties. Finally, we present the coding procedure and analysis of these two coding mechanisms (§ 4.5).

4.2 Data Characteristics

This subsection analyzes the data characteristics of workloads to show the potential of cold data encoding. Current 3D TLC NAND flash memories apply one-pass programming to write all three pages into a WL through a single programming operation [5, 37]. Therefore, we study the data characteristics in the WL granularity. We collect the distribution of voltage states of flash cells in each WL. Each WL contains a large number of flash cells, for example, 16×1024×8 cells in the flash chips we evaluated, with a page size of 16KB. Each cell is programmed into one of the 8 voltage states S_i, to store three data bits. For a simple illustration, we use decimal characters [7, 6, 4, 0, 2, 3, 1, 5] to represent the voltage states, by converting the three bits into decimals. We collected 20 workloads from different scenarios, and they could be divided into four groups based on the distribution of data characters. The first three groups are named $W-$, $L-$, and $U-$shaped distributions based on the line shape of the probability distribution. The fourth group is the Even (E) distribution. We select one from each group and present the results in Figure 6. The workload details can be found in Section 5.

We show that the combination of consecutive cells in some workloads exhibits skewed properties. One cell or a combination of several consecutive flash cells is considered a single entity to study the original data distribution of each WL. In this study, we have three settings. For the 1-cell, each flash cell is an entity. We count how many flash cells are in each of the eight voltage states in a WL, which can be regarded as eight characters. For 2-cell, we count the combination of characters from 2 consecutive cells (i.e., the combination of 2 characters). There are 64 combined characters, since each cell could be one of the eight states/characters. Similarly, the 4-cell case has a total of 4096 different combined state values (4096 combined characters). We sort the counts of the (combined) characters in descending order and convert them into ratios. Figure 7 shows the cumulative distribution function (CDF) of the ratios of combined characters. The X-axis represents the sorted index of combined characters based on the frequency. 1 in the X-axis represents the index of the most frequent combined character.

In the $E-$shaped workload, all the CDFs in the three figures are linear functions. For the other three types of workloads, the CDF of the 1-cell case is near a linear function, which means that flash cells in each WL tend to have an even distribution over 8 voltage states. Therefore, it is hard to construct some skewed data patterns by merely mapping the most frequent state to the desired voltage states, for example, to construct patterns with lots of cells in the lowest voltage state [13, 14, 23, 44, 57]. Those strategies would not work well, which could achieve 28.6% of the destination state at most. We should also notice that mapping to low voltage

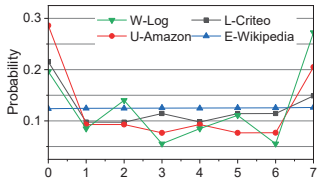
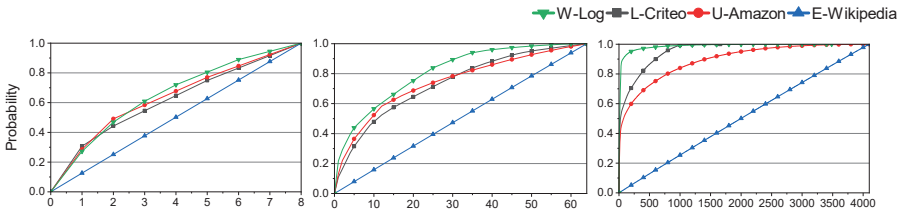


Figure 6. The distribution of 8 characters. One workload in each shape is presented.



(a) 1-cell **(b)** 2-cell **(c)** 4-cell

Figure 7. The CDF of (combined) characters. The X-axis is the index of the sorted sequence of (combined) characters, in descending order.

states does not benefit today's high-density 3D NAND flash, as presented above.

For these consecutive-cell cases of *L*-, *W*-, and *U*-shaped workloads, the CDFs of 2-cell and 4-cell are convex lines. The top 12.5% combined characters take up 46.2% and 48.3% among all characters for 2-cell and 4-cell, respectively. Therefore, the combination of consecutive cells results in some skewed properties, which present the potential for constructing the skewed data patterns presented in Section 3.

To deal with different data characteristics, we introduce separate coding schemes tailored for even and not-even (*L*-, *W*-, and *U*-shaped) data. Thus, our approach ensures optimal performance and reliability across a wide range of workloads, making it universally applicable to diverse real-world data.

4.3 Skewed Coding

Based on the above analysis, we first propose to construct skewed data patterns for data in not-even groups. In this work, the goal is to construct skewed data patterns dominated by certain 2 voltage states for TLC NAND flash memories, as the characterization in Section 3 shows two DS data patterns have high reliability.

Constant	20 02 00 22 21 01 23 03 24 04 25 25 26 06 27 07
Sorted characters	34 51 47 12 36 45 56 32 44 61 04 51 56 74 21 23

(a) 2-cell mapping

Constant	2020 0202 0022 2200 0220 2002 2220 2000 2202
Sorted characters	3451 4712 3645 5632 4461 0451 5674 2123 7246

(b) 4-cell mapping

Figure 8. The illustration of the skewed coding strategy.

Previous characterization shows that 2 DS data patterns (PS34, PS35) present low RBER in the evaluated TLC flash chip. The optimal data patterns can vary across different flash chips. For generalization, we use S_a and S_b to denote these two destination voltage states, whose combination forms the data pattern with low RBERs. The basic idea is to *map the frequent combination of consecutive voltage states to the combination of S_a and S_b*. Figure 8 illustrates the design, showing 2-cell and 4-cell mapping. Consecutive 2 or 4 cells are regarded as a single entity for mapping.

Suppose we want to construct a data pattern dominated by S3 (000) and S4(010), which corresponds to '0' and '2'. To introduce more '0' and '2', we map the combined characters that occur with high frequencies into the combinations of '0' and '2'. We count the frequencies of all combinations of consecutive 2 or 4 characters, and then sort the frequencies in descending order, represented as the numbers in black shown in Figure 8. We map them to the numbers in blue, which are set as constant patterns for a specific 3D NAND flash type. For example, in the 2-cell mapping, '34' has the highest frequency, and we map it to '20'. At the same time, we map '20' in the original data to '34' in the new coding. Similarly, suppose '3451' is the most frequent combination in the original data in a 4-cell combination, and we will map it to '2020'. With this simple mapping, the coded data would contain a large percentage of the desired states.

We then analyze the size and the storage location of the mapping table. For 2-cell mapping, there are 64 combinations, and each combination needs 6 bits. Thus, we need 24 bytes (64×6bits/2=192 bits) to store the mapping. In the tested flash chip, the space for user data on a page is 16KB, and the out-of-band (OOB) area is 2KB. The storage overhead of the mapping table for each WL (3 pages) is 0.05% (24B/48KB). All 24 bytes can be stored in the OOB area of each WL. The OOB area is usually used to store the ECC parity and the address mapping from the physical page address to the logical page address. Based on our evaluation, we found that the 2KB OOB area always has remaining space that is not used. Note that the numbers in blue are constant for a flash chip, and we don't need to store them for each WL.

For 4-cell mapping, there are 4096 combinations, and each combination needs 12 bits. We don't construct and store the full 4096 mapping for two reasons. First, the storage overhead of full mapping is high, which is 3KB for each WL. Second, not all 4096 combinations would occur in the data of a WL (the WL size is 48KB). As shown in Figure 7, the top 32 combined characters take up 42.9% of all possible combined characters. Therefore, we only map the first 32 combinations, which could already construct a data pattern dominated by '0' and '2'. The combined states that are not stored for mapping would stay in their original codes. In this case, the mapping table size is 48 bytes, which is 0.1% of the WL size. In the experiments, the default setting for the mapping size of 4-cell coding is 48 bytes.

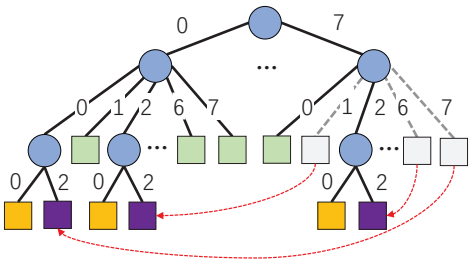


Figure 9. Illustration of the reversed Huffman coding. The original uniform-length (i.e., 2) characters are encoded into variable lengths to increase the frequencies of '0' and '2'. In this example, we encode '77' to '002' and '00' to '000'.

4.4 Reversed Huffman Coding

For evenly distributed data, we introduce a reversed Huffman coding mechanism to transform data into skewed patterns. With the same generalization in Section 4.3 (i.e., S_a and S_b), the basic idea is to *code the combinations without or with few S_a and S_b to newly constructed combinations. We construct those new combinations by appending S_a or S_b to the existing combination of S_a and S_b*. This coding makes a trade-off between the space (the coded data length) and data reliability (represented by the ratio of preferred characters).

Figure 9 shows an example of the proposed reversed Huffman coding mechanism, where two consecutive cells are combined as an entity for coding, i.e., 2-cell mapping. Again, suppose we aim to construct data patterns dominated by S3 and S4 (i.e., '0' and '2' as preferred characters). In the *E*−shaped data, each combined character has the same length (i.e., 2 cells) and a similar frequency. To increase the frequency of preferred characters, we first group a pair of combined characters, where one has no or few preferred characters (i.e., '77') and the other has lots of preferred characters (i.e., '00'). Then, we encode them into new combinations constructed by appending a preferred character. Two examples are: encoding '77' to '002' and '00' to '000'; encoding '71' to '022' and '02' to '020'. With this simple coding, there are more preferred characters, at the cost of increasing the total length of the coded data.

For 2-cell mapping, we note that there are only 4 combinations that have two preferred characters. Such coding combinations result in a limited increase in the ratio of preferred characters. The coding mechanism then groups the pairs with the combination that includes one preferred character. An example of such a pair is '76' and '72', where '76' is coded to '722' and '72' is coded to '720'. For each such pair, the sum ratio of '0' and '2' is increased by about 1.25%, and the total length is increased by about 1.56%.

Table 1 shows the achievable ratios of preferred characters and the space overheads with different numbers of pairs for mapping. The benefits are relatively limited with noticeable overhead when two cells are combined. To exploit a better tradeoff between benefits and overheads, we apply the above design principle to the 4-cell combination. For example, by grouping the pair of '7777' to '0000', we map '7777' to '00002' and the original '0000' to '00000'. With the 4-cell mapping, the achievable ratio of preferred characters is increased from the original 25% to 41.96%, with 9.86% space overhead, highlighted in green in Table 1. The other highlighted setting is to achieve 46.46% of preferred characters with 13.09% overhead, which will be evaluated in the experimental section.

Table 1. The achievable benefits (sum ratio of preferred characters) and overheads of reversed Huffman coding.

2-cell mapping							
Number of Pairs	0	4	8	16	20	24	28
Ratio of 0 & 2 (%)	25	35.29	41.67	52.50	57.14	61.36	65.22
Overhead (%)	0	6.25	12.5	25	31.25	37.5	43.75
4-cell mapping							
Number of Pairs	0	16	300	600	792	900	1056
Ratio of 0 & 2 (%)	25	25.54	32.50	38.22	41.96	43.55	46.46
Overhead (%)	0	0.20	3.66	7.32	9.86	10.99	13.09

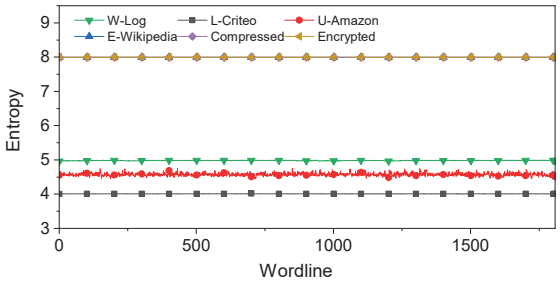


Figure 10. The data entropy in the granularity of WLs. The lines of *E*−shaped *Wikipedia*, *Compressed*, and *Encrypted* are overlapped, near value 8, the largest possible value.

4.5 Coding Procedure and Analysis

4.5.1 Implementation. The file system uses a custom NVMe feature to append the one-bit cold tag to write commands. This cold tag is transmitted to the SSD as part of the I/O request. The SSD controller uses it to decide whether to adopt the proposed coding strategy for programming. Upon programming, we select a coding strategy using the data entropy. Figure 10 shows the data entropy in WL granularity, which is distinctly separated into high or low values. The original plain files, such as texts, usually have low entropy values, and each file has a distinct entropy value over all WLs. The *E*−shaped *Wikipedia*, and compressed or encrypted files, presented as *Compressed* and *Encrypted* in Figure 10, have high entropy, very close to the largest possible entropy value 8. Therefore, we can select the coding by quickly estimating the data entropy. To ensure efficiency, we implement the entropy calculation using integer operations in controller-internal ARM cores with the aid of a small lookup table (less than 1KB) stored in the DRAM of the controller, which

reduces the calculation overhead to sub-microsecond levels [19]. This lightweight approach ensures that the entropy estimation does not introduce significant latency.

After the selection, we will code the data and write it to the corresponding blocks. To manage the coded data efficiently, we maintain two types of blocks in each flash die: one for skewed coding and another for reversed Huffman coding.

Procedure of skewed coding. In the skewed coding, we first count the frequency of combined characters in the original data. Our analysis reveals that data grouped by WL granularity within the same file exhibit a similar distribution of combined characters. Thus, we only calculate the frequency of combined characters for the first write request of the file. The subsequent data writes can reuse the same mapping table obtained from the first write. This optimization significantly reduces the computational overhead, as the frequency calculation is performed only once per file.

The time overhead for calculating the frequency is in the microsecond level, which is negligible compared to the write latency of TLC NAND flash (1500-5000 μs). Although all WLs storing data from the same file share one mapping table, we store the mapping table in each WL to facilitate independent decoding during read operations. This design ensures that each WL can be read and decoded without requiring access to mapping tables stored in other WLs, simplifying the decoding process. Finally, with the mapping table, the data is encoded/decoded in a manner similar to the randomizer used in commodity flash controllers. This approach does not require additional overhead in the firmware, as the existing randomizer already provides the necessary functionality for character transformation.

Procedure of reversed Huffman coding. In the reversed Huffman coding, we predefine a mapping table for all files, as the characters or combined characters exhibit an even distribution. This predefined mapping table with 4KB size, is designed to handle the uniform distribution of characters efficiently. During encoding, characters are transformed according to the predefined mapping, similar to the current randomizer. If the data distribution is ideally even, the overhead remains constant, as calculated in Table 1. However, in real-world scenarios, the distribution is not perfectly even, leading to a marginal increase in space overhead. To deal with the issue, we set the overhead slightly larger than the calculated value. For example, our experiments implement a case where the sum ratio of '0' and '2' is 41.96%, with a calculated overhead of 9.86%. To ensure robustness, we reserve an overhead of 10%. Extensive experiments confirm that no WL exceeds this reserved overhead. Similarly, for the case with a ratio of 46.46%, we reserve the overhead as 13.2%, where the calculated overhead is 13.09%. These reservations are sufficient to handle all evaluated files, including randomized data such as compressed and encrypted files.

For blocks using reversed Huffman coding, the data originally belonging to one WL may be stored across two physical WLs due to the coding overhead. This occurs because the constant-length coded data exceeds the size of a single WL. To manage this, we revisit the page translation table to record the order of data in each physical block. This approach ensures that the data can be accurately reconstructed during read operations, even when it spans two WLs. The index-based management strategy is lightweight and introduces minimal overhead, as it only requires storing additional indexes in the translation table.

4.5.2 Analysis.

We quantify lifetime impacts using logical capacity and discuss performance implications.

Logical capacity. The proposed coding strategy is a double-edged sword to the SSD lifetime. On one hand, the reversed Huffman coding strategy introduces space overhead, which consumes space for user data. On the other hand, the RBER is reduced, which means the same physical drive can endure a larger number of P/E cycles, and the refresh frequency is reduced. The lifetime of NAND flash, represented as logical written capacity, can be calculated as the following equation.

$$lifetime = \frac{PE \times (1 + OP)}{365 \times DWPD \times WA \times (1 + OC)} \quad (1)$$

OP, WA, $DWPD$, and PE are the over-provision factor, write amplification factor, the volume of data written per day, and the available P/E cycles of blocks, respectively. OC is the overhead of the proposed coding schemes.

In skewed coding, OC is 0, but it consumes 48 bytes of ECC parity per WL. Each codeword in a WL contains 2KB data and 224B parity, resulting in a 2B parity loss per codeword, reducing ECC capacity by less than 3%. While this may necessitate an increased WA due to higher refresh frequency, the lower RBER of skewed-encoded data significantly reduces the refresh frequency, outweighing the ECC capacity loss.

The reversed Huffman coding can preserve the ECC capability, but increase the OC. OC has two settings in this work (i.e., 10% and 13.2%). The experiments will show that although the coding imposes non-negligible overheads, the lifetime could be improved because of the write amplification reduction from less data refresh.

Performance impacts. The coding strategies encode the data of a WL before programming, which also requires reading out all three pages in the WL for decoding. For small-granularity read requests, the performance would be impacted, as the latency increases from one page to one WL. Targeting cold data, the performance impact is marginal. On one hand, cold data are rarely accessed and thus contribute little to the overall access performance. On the other hand, cold data tend to be accessed in large granularity [48, 52]. For example, archived data are usually not read. Once needed, all data in the same file might be read out. In this case, our method introduces no additional read overhead, as all pages in the WL are required.

5 Experimental Evaluations

5.1 Experimental Setup

The proposed approach is evaluated on the YEESTOR 9081 and 9086 SSD evaluation platform [50]. The evaluated flash chips in this section are from Micron, with 176 layers in a block. The number of P/E cycles is set as 1000, and flash chips are baked with high temperatures at 100°C for simulating long retention time [26, 28, 30].

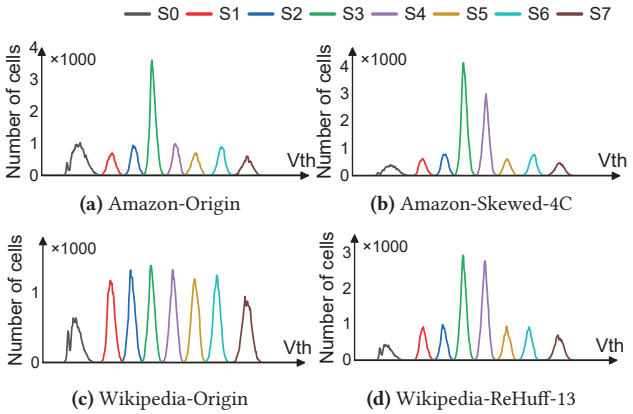


Figure 11. The V_{th} distribution of original and encoded data.

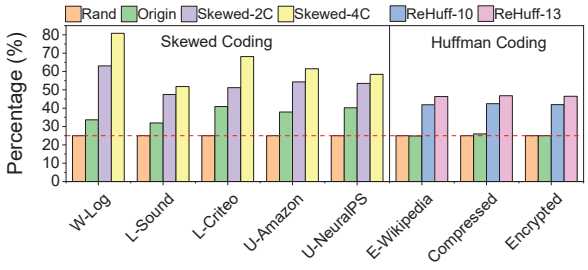


Figure 12. The sum ratio of S3 and S4. The dashed line points to 25%, which marks the sum ratio of randomized data.

The following strategies are evaluated on 3D flash chips.

- **Rand**: Data is randomized before programming, which is the representative strategy in most 3D flash devices.
- **Origin**: The original raw data without coding.
- **Skewed-2C**: The 2-cell coding regarding consecutive 2 cells as an entity.
- **Skewed-4C**: The 4-cell coding regarding consecutive 4 cells as an entity.
- **ReHuff-10**: The reversed Huffman coding with 9.86% overhead, which is set to 10% for reservation.
- **ReHuff-13**: The reversed Huffman coding with 13.09% overhead, which is set to 13.2% for reservation.

The skewed coding and reversed Huffman coding are to construct data dominated by '0' and '2'. This setting is because the data pattern dominated by S3 (000, i.e., 0) and S4 (010, i.e., 2) has very low RBER in the tested flash chip. We evaluate the reliability enhancement on six open workloads: *W*−shaped *Log* [24], *L*−shaped *Sound* [40] and *Criteo* [9], *U*−shaped *Amazon* [10] and *NeuralPS* [11], and *E*−shaped *Wikipedia* [12]. The skewed coding is evaluated on the first five workloads. The reversed Huffman coding is evaluated on *E*−shaped *Wikipedia*. For reversed Huffman coding, we also generate two files by compressing and encrypting *Amazon*, respectively, presented as *Compressed* and *Encrypted*.

5.2 Experimental Results

State distribution: We first present the state distribution of two workloads, one from the uneven distributions and one from even distributions, as shown in Figure 11. In this example, the destination voltage states are S3 and S4, that is to construct patterns with lots of '0' and '2'. The original distribution in *Amazon* looks like data pattern `PS03` introduced in Section 3, with lots of cells in S0 and S3, shown in Figure 11a. The skewed coding strategy successfully encodes the data to the data pattern dominated by '0' and '2'. With the reversed Huffman coding, the even distribution of *Wikipedia* in Figure 11c is changed to skewed patterns.

Improvement in the ratio of preferred characters: Figure 12 further shows the sum percentage of 0 and 2 to quantitatively estimate the effectiveness of ColdCode. With randomization, each state takes about 12.5%, and the sum ratio of 0 and 2 is about 25%. In the original workloads, except *Wikipedia*, the sum ratio of 0 and 2 is 38%, on average. *Skewed-2C* and *Skewed-4C* could effectively increase the ratio to 54% and 58%, respectively. Recall that the ratio of the manually-constructed data pattern `PS34` is 62.5%, which has the lowest RBER among all the constructed patterns. Our method achieves a similar ratio to `PS34`, and we will present that our coded data have similar reliability to `PS34`. In the case of even distribution, including *Wikipedia*, the compressed and encrypted data, the sum ratio of 0 and 2 in the original data is around 25%, very similar to that of randomized data. After the reversed Huffman coding with 10% and 13.2% overheads, the ratios are about 42% and 47%, respectively. It is worth noting that these values are very close to the values that are calculated in Table 1. This is because the data being sliced into the WL granularity also have nearly identical even distributions.

Average RBER reduction: Figure 13 shows the RBER comparison, presenting the average value of all WLs in a flash block, after baking at 100°C for 4 hours. Because *Rand* performs the same under different workloads, we only present one set of results from *Rand*. The left part in Figure 13 evaluates the RBER reduction of the skewed coding. As workloads have different distributions in their original data, *Origin* of some workloads might have worse reliability than *Rand*. Nevertheless, the proposed skewed coding significantly reduces the number of bit errors. Compared to *Origin*, *Skewed-2C* and *Skewed-4C* could averagely reduce the number of bit errors in the MSB page by 30% and 42%, respectively. The right part in Figure 13 evaluates the RBER reduction of the reversed

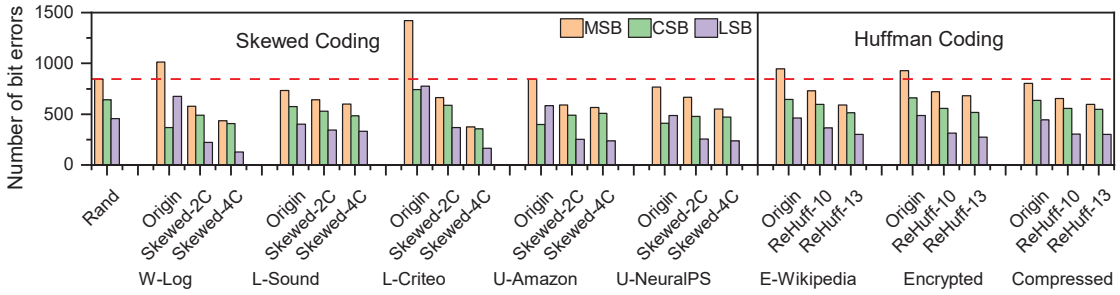
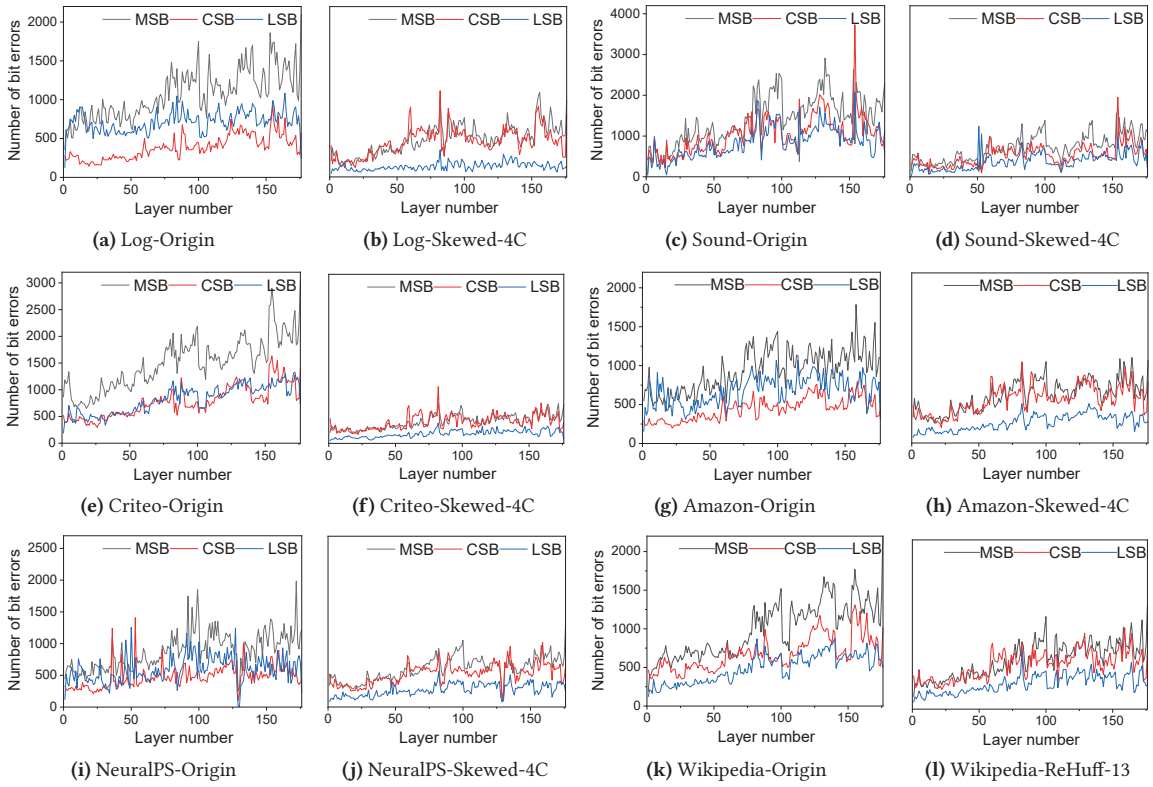


Figure 13. The page RBER comparison among different strategies.

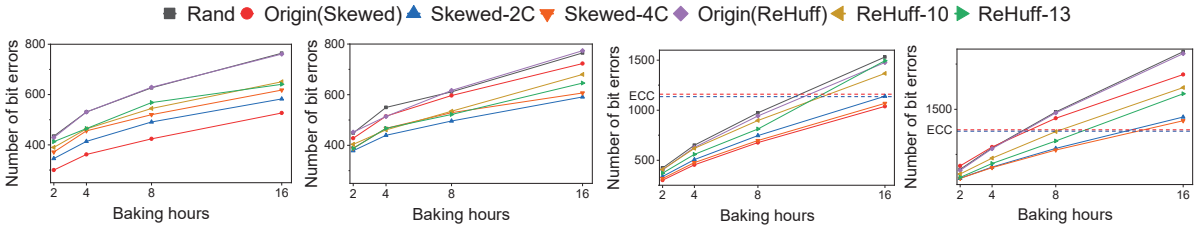


(a) Log-Origin **(b)** Log-Skewed-4C **(c)** Sound-Origin **(d)** Sound-Skewed-4C

(e) Criteo-Origin **(f)** Criteo-Skewed-4C **(g)** Amazon-Origin **(h)** Amazon-Skewed-4C

(i) NeuralPS-Origin **(j)** NeuralPS-Skewed-4C **(k)** Wikipedia-Origin **(l)** Wikipedia-ReHuff-13

Figure 14. The comparison of the number of bit errors at different layers.



(a) CSB page with optimal read **(b)** MSB page with optimal read **(c)** CSB page with default read **(d)** MSB page with default read

Figure 15. The increase of the page RBER over baking hours. Figures a-b present the max number of bit errors by reading with the optimal read voltages, while Figures c-d present the average number of bit errors by reading the default read voltages. The dashed red lines are the original ECC capacity, and the dashed blue lines are the lowered ECC capacity in skewed coding.

Huffman coding. *Origin* has similar reliability as *Rand* due to the near-even distribution. The reversed Huffman coding with 10% and 13.2% overhead averagely reduces the number of bit errors in MSB pages by 21% and 30%, respectively.

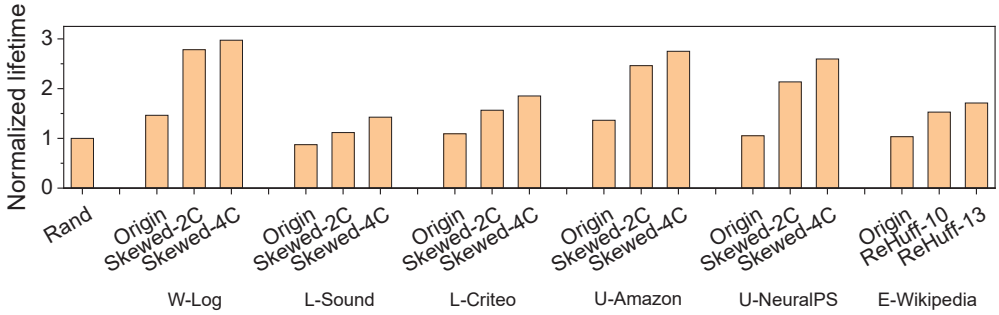


Figure 16. The lifetime improvement.

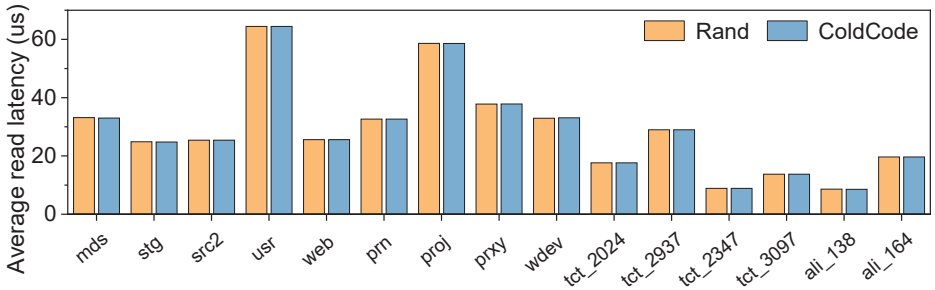


Figure 17. The read performance comparison.

Overall reliability enhancement: The above shows the reduction of the average bit errors. We then present the page RBER in each layer, shown in Figure 14. Because of the cross-layer variations in 3D NAND [17], the RBER difference could exceed one order of magnitude in *Origin*. For example, the MSB page of wordline 1256 has 1784 bit errors, compared to 80 bit errors on the LSB page of wordline 6, in Figure 14g. Such variations are well relieved by ColdCode. Both the variations among wordlines and among page types are significantly reduced in all workloads. Due to the existence of variations, error correction mechanisms or reliability guarantee strategies are designed for the worst-case RBER in NAND flash memories. The reduction of variation could reduce the reliability guarantee costs, which is especially important for high-density 3D NAND flash.

Lifetime improvement. As presented in Section 4.5, the lifetime can be improved due to the reduction in write amplification. This work mainly considers the write amplification caused by data refresh [21, 29, 33]. Data refresh is necessary to mitigate retention errors, caused by charge loss over time, especially for cold data as data remain unchanged for extended periods. However, frequent refresh operations exacerbate write amplification. Figure 15 shows how the number of bit errors increases with the baking hours at 100°C. Each point in Figures 15a-15b is the max number of bit errors of all pages within a flash block, by reading data with the optimal read voltages, that leads to the least number of bit errors [25, 35, 39]. The results show that our strategies introduce lower RBER than *Rand* and *Origin*. Our methods can be integrated with state-of-the-art voltage tuning techniques [25, 35, 39] to reduce RBER. Since attaining absolute optimal values is challenging in practice, we provide average bit error rates by reading with default read voltages, as shown in Figures 15c-15d. When the average RBER approaches the ECC capability, numerous pages experience substantial read retries [8, 49], prompting the controller to initiate data refresh. For performance considerations, we use the average RBER with the default read voltages as the reliability metric. *Rand* and *Origin* of *Wikipedia* perform the worst on MSB pages. The raw bit errors reach the ECC capability after baking for 8.35 hours on MSB pages, equivalent to 2.4 months at 40°C [4, 47]. Note that the ECC capacity of *Skewed-4C* is slightly lower than the original ECC capability due to the occupation of the mapping table in OOB. Nevertheless, *Skewed-4C* prolongs the refresh period to 5.58 months, which is 2.29 times of *Rand*.

We conduct simulations on SimpleSSD [20, 55] to study the lifetime impacts. We implement and simulate refresh mechanisms in real workloads over different coding strategies. Finally, we transform the write amplification data into lifetime based on Equation 1, as shown in Figure 16. Compared to *Rand*, the skewed coding prolongs the lifetime by 1.42×-2.97×, while the reversed Huffman coding prolongs lifetime by 1.7×.

Performance impact. We compare the read performance between *Rand* and *CodeCold*. Because the datasets in the above evaluations only contain data bytes, without I/O requests information, we need proper performance evaluation workloads. We introduce MSRC [34], TencentCloud [58], and

AliCloud [1] to simulate the coding status before and after *CodeCold*. Figure 17 shows the read performance comparison. The results show that *CodeCold* performs very similarly to *Rand*, indicating that *CodeCold* introduces little impact on the read performance of normal data access. The reason is that the proposed strategy only encodes cold data, while hot data coding remains unchanged from the previous design. Thus, the performance of hot data is not impacted at all. For cold data, they are rarely accessed and contribute little to the overall performance. In addition, cold data tends to be accessed in large granularity, e.g., to load an archived file, where all pages in one WL are required. In this case, our method introduces no additional read overhead.

6 Conclusions

High RBER of cold data is one of the most critical challenges for current high-density 3D NAND flash memories. To deal with extreme cases, randomization is widely adopted in real 3D flash chips. This paper presents that randomization is not the best practice for cold data storage in high-density 3D flash. We propose to replace the randomizer in 3D NAND flash with a new cold data encoding framework. Through experiments on real devices, we show that the new coding strategies could largely reduce the RBER of stored data on arbitrary data types, thus prolonging the flash lifetime due to the reduction in data refresh frequency.

Acknowledgments

We sincerely thank the shepherd and anonymous reviewers for their constructive feedback. This work is supported in part by the National Key Research and Development Program of China under Grant No. 2023YFB4502702 and the National Natural Science Foundation of China under Grant No. 62332021 and 62472007. The corresponding author is Jie Zhang (jiez@pku.edu.cn).

References

[1] 2018. Alibaba Block Traces. https://github.com/alibaba/block-traces.

[2] Tobias Ahrens, Mohammed Rajab, and Jürgen Freudenberger. 2016. Compression of short data blocks to improve the reliability of non-volatile flash memories. In *2016 International Conference on Information and Digital Technologies (IDT)*. IEEE, 1–4.

[3] Yu Cai, Ning Chen, Yunxiang Wu, Erich F Haratsch, Earl T Cohen, and Timothy L Canepa. 2016. Method to distribute user data and error correction data over different page types by leveraging error rate variations. US Patent 9,262,268.

[4] Yu Cai, Saugata Ghose, Erich F Haratsch, Yixin Luo, and Onur Mutlu. 2017. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives. *Proc. IEEE* 105, 9 (2017), 1666–1704.

[5] Yu Cai, Saugata Ghose, Yixin Luo, Ken Mai, Onur Mutlu, and Erich F Haratsch. 2017. Vulnerabilities in MLC NAND flash memory programming: Experimental analysis, exploits, and mitigation techniques. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 49–60.

[6] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. 2012. Error patterns in MLC NAND flash memory: Measurement, characterization, and analysis. In *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 521–526.

[7] Wonil Choi, Myoungsoo Jung, and Mahmut Kandemir. 2018. Invalid data-aware coding to enhance the read performance of high-density flash memories. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 482–493.

[8] Myoungjun Chun, Jaeyong Lee, Myungsuk Kim, Jisung Park, and Jihong Kim. 2024. RiF: Improving Read Performance of Modern SSDs Using an On-Die Early-Retry Engine. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 643–656.

[9] Criteo Dataset. 2014. *https://ailab.criteo.com/ressources/*.

[10] Kaggle Datasets. 2017. Amazon Reviews for Sentiment Analysis. https://www.kaggle.com/datasets/bittlingmayer/amazonreviews.

[11] Kaggle Datasets. 2017. NIPS Papers. https://www.kaggle.com/datasets/benhamner/nips-papers.

[12] Kaggle Datasets. 2018. Wikipedia Sentences. https://www.kaggle.com/datasets/mikeortman/wikipedia-sentences.

[13] Masafumi Doi, Shuhei Tanakamaru, and Ken Takeuchi. 2014. A scaling scenario of asymmetric coding to reduce both data retention and program disturbance of NAND flash memories. *Solid-State Electronics* 92 (2014), 63–69.

[14] Jie Guo, Danghui Wang, Zili Shao, and Yiran Chen. 2017. Data-pattern-aware error prevention technique to improve system reliability. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 4 (2017), 1433–1443.

[15] Jie Guo, Wujie Wen, Jingtong Hu, Danghui Wang, Hai Li, and Yiran Chen. 2015. FlexLevel: A novel NAND flash storage system design for LDPC latency reduction. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6.

[16] Tsutomu Higuchi, Takuyo Kodama, Koji Kato, Ryo Fukuda, Naoya Tokiwa, Mitsuhiro Abe, Teruo Takagiwa, et al. 2021. 30.4 A 1Tb 3b/Cell 3D-Flash Memory in a 170+ Word-Line-Layer Technology. In *Proceedings of the 68th IEEE International Solid- State Circuits Conference (ISSCC)*, Vol. 64. doi:10.1109/ISSCC42613.2021.9366003

[17] Duwon Hong, Myungsuk Kim, Geonhee Cho, Dusol Lee, and Jihong Kim. 2022. GuardedErase: Extending SSD Lifetimes by Protecting Weak Wordlines. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 133–146.

[18] Shehbaz Jaffer, Kaveh Mahdaviani, and Bianca Schroeder. 2022. Improving the Reliability of Next Generation SSDs using WOM-v Codes. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 117–132.

[19] Cheng Ji, Li-Pin Chang, Liang Shi, Congming Gao, Chao Wu, Yuangang Wang, and Chun Jason Xue. 2017. Lightweight data compression for mobile flash storage. *ACM Transactions on Embedded Computing Systems (TECS)* 16, 5s (2017), 1–18.

[20] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. 2017. SimpleSSD: Modeling solid state drives for holistic system simulation. *IEEE Computer Architecture Letters* 17, 1 (2017), 37–41.

[21] Bryan S. Kim, Jongmoo Choi, and Sang Lyul Min. 2019. Design Tradeoffs for SSD Reliability. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*. Boston, MA, 281–294.

[22] Changman Lee, Dongho Sim, Jooyoung Hwang, and Sangyeun Cho. 2015. {F2FS}: A new file system for flash storage. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*. 273–286.

[23] Wonyoung Lee, Mincheol Kang, Seokin Hong, and Soontae Kim. 2019. Interpage-based endurance-enhancing lower state encoding for MLC and TLC flash memory storages. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 9 (2019), 2033–2045.

[24] Leveldb. 2014. *https://github.com/google/leveldb*.

[25] Qiao Li, Min Ye, Yufei Cui, Liang Shi, Xiaoqiang Li, Tei-Wei Kuo, and Chun Jason Xue. 2020. Shaving retries with sentinels for fast read over

high-density 3d flash. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 483–495.

[26] Qiao Li, Min Ye, Tei-Wei Kuo, and Chun Jason Xue. 2021. How the common retention acceleration method of 3D NAND flash memory goes wrong?. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems*. 1–7.

[27] Yu Liang, Riwei Pan, Tianyu Ren, Yufei Cui, Rachata Ausavarungnirun, Xianzhang Chen, Changlong Li, Tei-Wei Kuo, and Chun Jason Xue. 2022. {CacheSifter}: Sifting cache files for boosted mobile performance and lifetime. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 445–459.

[28] Weihua Liu, Fei Wu, Xiang Chen, Meng Zhang, Yu Wang, Xiangfeng Lu, and Changsheng Xie. 2022. Characterization Summary of Performance, Reliability, and Threshold Voltage Distribution of 3D Charge-Trap NAND Flash Memory. *ACM Transactions on Storage (TOS)* 18, 2 (2022), 1–25.

[29] Yixin Luo, Yu Cai, Saugata Ghose, Jongmoo Choi, and Onur Mutlu. 2015. WARM: Improving NAND flash memory lifetime with write-hotness aware retention management. In *2015 31st Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 1–14.

[30] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. 2018. HeatWatch: Improving 3D NAND flash memory device reliability by exploiting self-recovery and temperature awareness. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 504–517.

[31] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. 2018. Improving 3D NAND flash memory lifetime by tolerating early retention loss and process variation. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2, 3 (2018), 1–48.

[32] Stathis Maneas, Kaveh Mahdaviani, Tim Emami, and Bianca Schroeder. 2022. Operational Characteristics of SSDs in Enterprise Storage Systems: A Large-Scale Field Study. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 165–180.

[33] Vidyabhushan Mohan, Sriram Sankar, Sudhanva Gurumurthi, and W Redmond. 2012. reFresh SSDs: Enabling high endurance, low cost flash in datacenters. *Univ. of Virginia, Tech. Rep. CS-2012-05* (2012).

[34] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. 2008. Write off-loading: Practical power management for enterprise storage. *ACM Transactions on Storage (TOS)* 4, 3 (2008), 1–23.

[35] Nikolaos Papandreou, Thomas Parnell, Haralampos Pozidis, Thomas Mittelholzer, Evangelos Eleftheriou, Charles Camp, Thomas Griffin, Gary Tressler, and Andrew Walls. 2015. Enhancing the reliability of MLC NAND flash memory systems by read channel optimization. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 20, 4 (2015), 1–24.

[36] Nikolaos Papandreou, Haralampos Pozidis, Thomas Parnell, Nikolas Ioannou, Roman Pletka, Sasa Tomic, Patrick Breen, Gary Tressler, Aaron Fry, and Timothy Fisher. 2019. Characterization and analysis of bit errors in 3D TLC NAND flash memory. In *2019 IEEE International Reliability Physics Symposium (IRPS)*. IEEE, 1–6.

[37] Jisung Park, Myungsuk Kim, Myoungjun Chun, Lois Orosa, Jihong Kim, and Onur Mutlu. 2021. Reducing solid-state drive read latency by optimizing read-retry. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 702–716.

[38] Ted Pekny, Luyen Vu, Jeff Tsai, Dheeraj Srinivasan, Erwin Yu, J. E. Pabustan, Joe Xu, et al. 2022. A 1-Tb Density 4b/Cell 3D-NAND Flash on 176-Tier Technology with 4-Independent Planes for Read using CMOS-Under-the-Array. *International Solid-State Circuits Conference (ISSCC)* 65 (2022), 1–3.

[39] Borja Peleato, Rajiv Agarwal, John M Cioffi, Minghai Qin, and Paul H Siegel. 2015. Adaptive read thresholds for NAND flash. *IEEE Transactions on Communications* 63, 9 (2015), 3069–3081.

[40] Karol J Piczak. 2015. ESC: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM international conference on Multimedia*. 1015–1018.

[41] Liang Shi, Longfei Luo, Yina Lv, Shicheng Li, Changlong Li, and Edwin Hsing-Mean Sha. 2021. Understanding and Optimizing Hybrid SSD with High-Density and Low-Cost Flash Memory. In *2021 IEEE 39th International Conference on Computer Design (ICCD)*. 236–243.

[42] Shun Suzuki, Hiroki Aihara, and Ken Takeuchi. 2020. Privacy protection NAND flash system with flexible data-lifetime control by in-3-D vertical cell processing. *IEEE Journal of Solid-State Circuits* 55, 10 (2020), 2802–2809.

[43] Debao Wei, Zhelong Piao, Hua Feng, Liyan Qiao, Cong Hu, and Xiyuan Peng. 2022. TCSE: A Target Cell States Elimination Coding Strategy for Highly Reliable Data Storage based on 3D NAND Flash Memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2022).

[44] Debao Wei, Liyan Qiao, Shiyuan Wang, and Xiyuan Peng. 2016. Fixation ratio of error location-aware strategy for increased reliable retention time of flash memory. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 24, 10 (2016), 3145–3155.

[45] Chin-Hsien Wu, Hao-Wei Zhang, Chia-Wei Liu, Ta-Ching Yu, and Chi-Yen Yang. 2021. A dynamic Huffman coding method for reliable TLC NAND flash memory. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 26, 5 (2021), 1–25.

[46] Wan-Ling Wu, Jen-Wei Hsieh, and Hao-Yu Ku. 2023. CDS: Coupled Data Storage to Enhance Read Performance of 3D TLC NAND Flash Memory. *IEEE Trans. Comput.* (2023).

[47] Qin Xiong, Fei Wu, Zhonghai Lu, Yue Zhu, You Zhou, Yibing Chu, Changsheng Xie, and Ping Huang. 2018. Characterizing 3D floating gate NAND flash: Observations, analyses, and implications. *ACM Transactions on Storage (TOS)* 14, 2 (2018), 1–31.

[48] Gala Yadgar, MOSHE Gabel, Shehbaz Jaffer, and Bianca Schroeder. 2021. SSD-based workload characteristics and their performance implications. *ACM Transactions on Storage (TOS)* 17, 1 (2021), 1–26.

[49] Min Ye, Qiao Li, Yina Lv, Jie Zhang, Tianyu Ren, Daniel Wen, Tei-Wei Kuo, and Chun Jason Xue. 2024. Achieving Near-Zero Read Retry for 3D NAND Flash Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 55–70.

[50] YEESTOR. 2023. Ys9083xt/ys9081xt ssd platform. https://www.yeestor.com/products/detail/i-42.html.

[51] Jui-Nan Yen, Yao-Ching Hsieh, Cheng-Yu Chen, Tseng-Yi Chen, Chia-Lin Yang, Hsiang-Yun Cheng, and Yixin Luo. 2022. Efficient Bad Block Management with Cluster Similarity. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 503–513.

[52] David Kuang-Hui Yu and Jen-Wei Hsieh. 2019. A Management Scheme of Multi-Level Retention-Time Queues for Improving the Endurance of Flash-Memory Storage Devices. *IEEE Trans. Comput.* 69, 4 (2019), 549–562.

[53] David Kuang-Hui Yu and Jen-Wei Hsieh. 2021. Differential Evolution Algorithm with Asymmetric Coding for Solving the Reliability Problem of 3D-TLC CT Flash Memory Storage Systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2021).

[54] Ta-Ching Yu, Chin-Hsien Wu, and Yan-Qi Liao. 2022. CRRC: Coordinating Retention Errors, Read Disturb Errors and Huffman Coding on TLC NAND Flash Memory. *IEEE Transactions on Dependable and Secure Computing* (2022).

[55] Jie Zhang and Myoungsoo Jung. 2020. Zng: Architecting gpu multi-processors with new flash for scalable data analysis. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 1064–1075.

[56] Meng Zhang, Fei Wu, Qin Yu, Weihua Liu, Yifan Wang, and Changsheng Xie. 2020. Exploiting error characteristic to optimize read voltage for 3-d NAND flash memory. *IEEE Transactions on Electron Devices* 67, 12 (2020), 5490–5496.

[57] Wenhui Zhang, Qiang Cao, and Zhonghai Lu. 2018. Bit-Flipping schemes upon MLC flash: Investigation, implementation, and evaluation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 4 (2018), 780–784.

[58] Yu Zhang, Ping Huang, Ke Zhou, Hua Wang, Jianying Hu, Yongguang Ji, and Bin Cheng. 2020. {OSCA}: An {Online-Model} Based Cache Allocation Scheme in Cloud Block Storage Systems. In *2020 USENIX Annual Technical Conference (ATC 20)*. 785–798.

[59] Yutong Zhao, Wei Tong, Jingning Liu, Dan Feng, and Hongwei Qin. 2019. CeSR: A Cell State Remapping Strategy to Reduce Raw Bit Error Rate of MLC NAND Flash.. In *MSST*. 161–171.